MetroLift's RAG Scope Fight

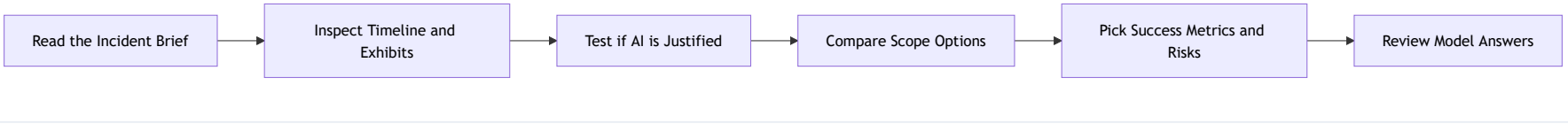
Table of Contents

⏱ **Duration: 30 minutes**

Learning Objectives

- Define a buildable AI-native use case brief that names consumers, constraints, risks, and measurable success criteria.
- Choose between a governed RAG platform scope and simpler alternatives using evidence from consumer needs, risks, and operating targets.
- Translate business, AI, and platform goals into distinct metrics with targets, owners, and evidence sources.
- Defend scope boundaries, deferred items, and mitigations that keep a capstone realistic without hiding critical risks.

Case Pipeline



Thursday, 4:40 p.m., and the demo is about to go off the rails

On **Sep 19, 2024**, MetroLift's capstone review turned into a fight. Priya Nandakumar, the staff data engineer leading the project, had ten weeks of work behind her and one slide in front of the review panel: **'Field Operations Copilot'**. The VP loved the phrase. The platform architect did not. He circled three words on the whiteboard: *copilot*, *real time*, *actionable*. Then he asked the question Priya had been avoiding: **what exactly are we building, for whom, and how will we know it works?**

MetroLift services elevators and escalators across 14 North American cities. The frontline problem is expensive and ugly: technicians lose time during outages because answers live in scattered manuals, service bulletins, PDF scan packs, and inconsistent incident notes. A downtown hospital outage the week before had taken **96 minutes** to resolve; the post-incident interview showed **19 minutes** were spent searching for the correct reset procedure after a firmware-specific bulletin overrode the base manual.

The decision in the room was not whether AI was fashionable. It was narrower and harder: **should MetroLift scope the capstone**

as a governed RAG assistant for technicians, back down to a simpler search-and-dashboard platform, or push forward into an agent-like operations copilot? The wrong choice would either produce an underpowered tool nobody used or a bloated architecture that could not be governed, tested, or finished.

What the team learned the hard way before the review

- **Jul 08, 2024** — Priya's team interviewed 11 field technicians and 4 service supervisors; technicians described 'document hunting' during outages, while supervisors emphasized trend visibility and SLA reporting.
- **Jul 19, 2024** — The first project brief proposed a broad 'AI operations assistant' covering troubleshooting, dispatch suggestions, parts recommendations, and work-order drafting.
- **Aug 02, 2024** — A feasibility spike ingested 1,200 PDFs and 75,000 CMMS ticket rows; retrieval over manuals looked promising, but action workflows lacked approval paths and audit requirements.
- **Aug 16, 2024** — Security flagged that 8.3% of incident notes contained names, phone numbers, or building access details, forcing PII redaction into the design.
- **Aug 27, 2024** — The team ran a shadow study across 32 outage cases and measured that technicians opened a median **4.7 documents** per case before acting.
- **Sep 06, 2024** — Priya drafted a risk register with owners across data quality, retrieval grounding, access control, cost, and schedule; automated dispatch remained unmitigated.
- **Sep 19, 2024** — The architecture review demanded a final scope decision tied to consumers, metrics, and what would be explicitly deferred.

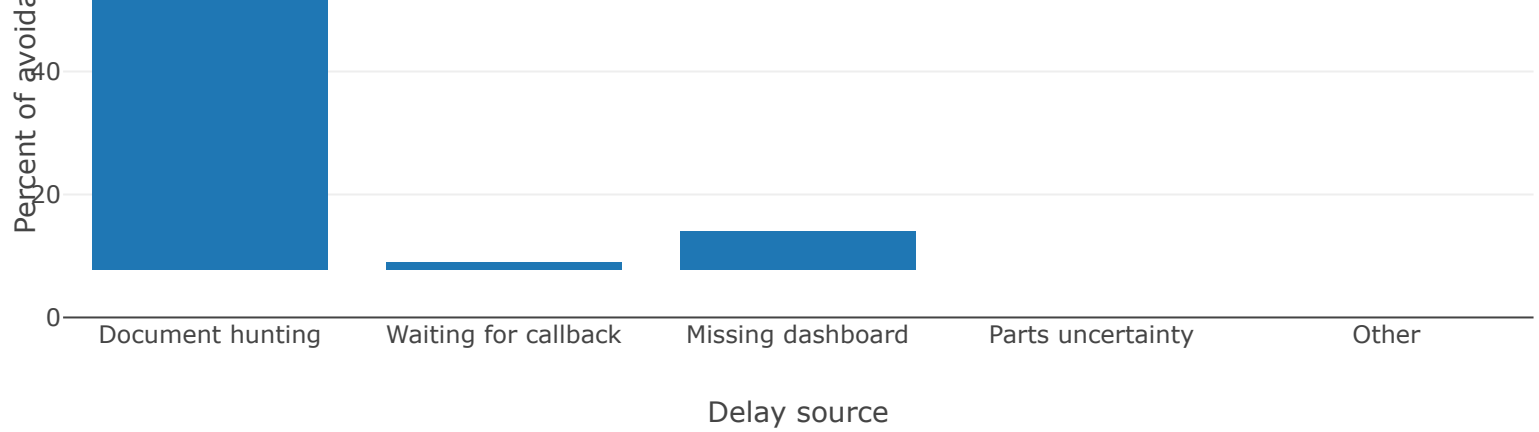
Exhibits from the review packet

Priya's best evidence was not architectural. It was operational.

Exhibit	Signal	Number	Why it changed the argument

Shadow study sample	Outage cases observed	32 cases	Large enough to expose recurring technician workflow pain
Median troubleshooting time	From arrival to confident next step	11 min 40 sec	Established baseline for the primary consumer
Documents opened per case	Manuals, bulletins, notes	4.7 median	Showed synthesis burden, not just lookup burden
Delay caused by document hunting	Share of avoidable delay	68%	Strongest evidence that unstructured retrieval is central
Delay caused by missing summary dashboards	Share of avoidable delay	14%	Undercut the case for a dashboard-only solution
Incident notes with sensitive details	PII or access information	8.3%	Forced governance controls into scope
Pilot corpus size	Manual pages + bulletins + summaries	38,400 + 2,100 + 18,600 docs	Big enough to justify retrieval design, small enough to build
Review timeline remaining	Weeks before capstone sign-off	10 weeks	Killed fantasies of building every production feature

Where technician delay came from in the 32-case shadow study

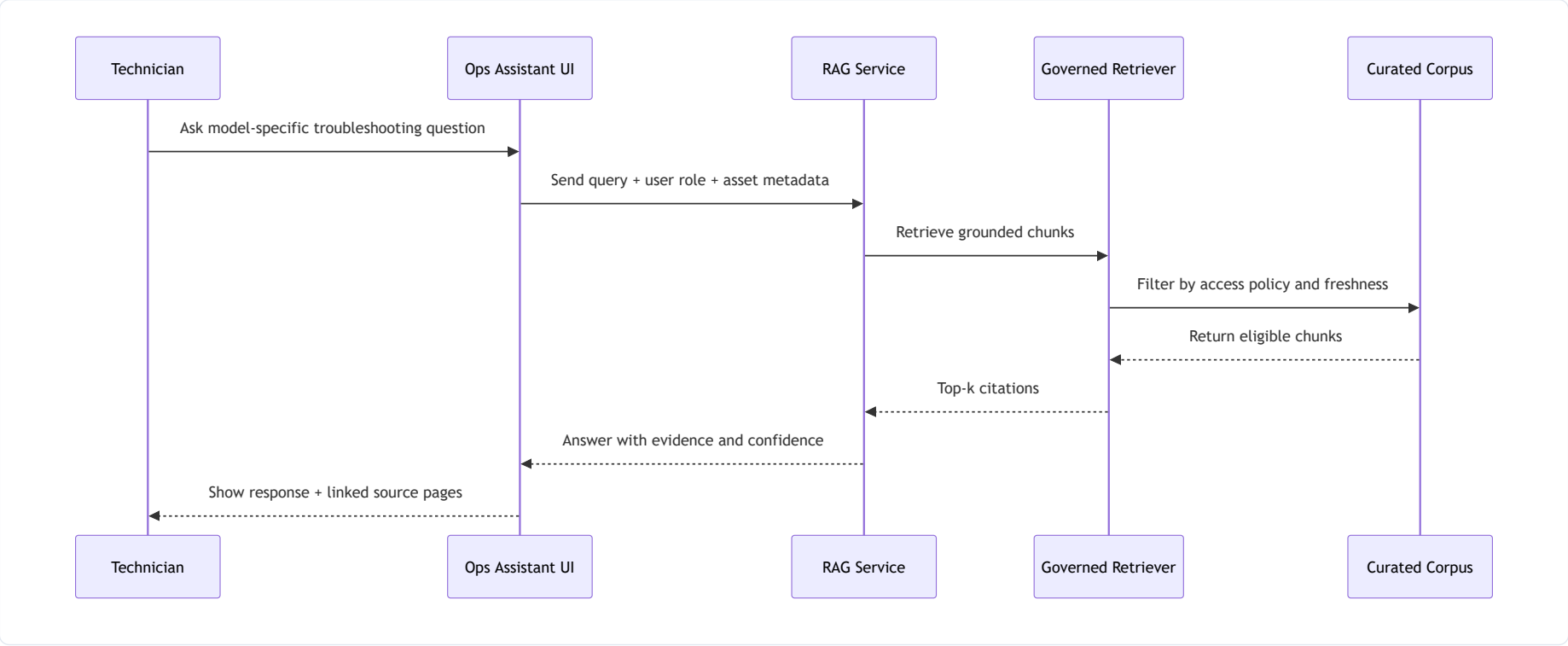


The review packet also included the proposed metric cut-down Priya wanted to enforce:

Goal bucket	Metric	Target	Owner	Evidence source
Business	Median time to confident next step	**< 5 min** in pilot	Service Ops PM	Time-and-motion study
Business	Technician self-reported usefulness	**≥ 75%** useful/very useful	Field Enablement Lead	Pilot survey
AI	Citation coverage	**≥ 90%** answers with at least one valid citation	ML engineer	Eval harness
AI	Groundedness score	**≥ 0.85** on adjudicated eval set	ML engineer	Weekly evaluation run
Platform	Corpus freshness for bulletins	**< 24 h** from source release	Data engineer	Pipeline audit table

Platform	p95 end-to-end latency	**< 3.5 s**	Platform engineer	Tracing dashboard
Platform	Failed ingestion runs	**< 2%** weekly	Data engineer	Orchestrator logs

The architecture sketch that survived the review was intentionally plain:



Questions to answer before you read the analysis

1. **[Analyze]** Which exhibit most strongly supports an AI-native RAG design over a simpler dashboard or keyword search tool, and why?
2. **[Evaluate]** Which of the three decision options best fits the ten-week capstone window without hiding essential risk, and what would you explicitly defer?
3. **[Apply]** Pick one metric from each goal bucket — business, AI, and platform — and explain how that trio would shape

architecture choices.

4. **[Analyze]** Which risk should change the design immediately rather than be parked for later: PII in notes, latency pressure, or feature ambition? Use the review evidence.

5. **[Evaluate]** If the team removed visible citations to cut latency below 2 seconds, would that improve the project or weaken it?

Model answers Priya wished the team had written first

For **Question 1**, the strongest exhibit is the **68% avoidable delay from document hunting** combined with the **4.7 documents opened per case**. That pair shows the core problem is not missing charts; it is cross-document retrieval and synthesis under pressure. That is the cleanest argument for Option 1 and the cleanest rebuttal to Option 2.

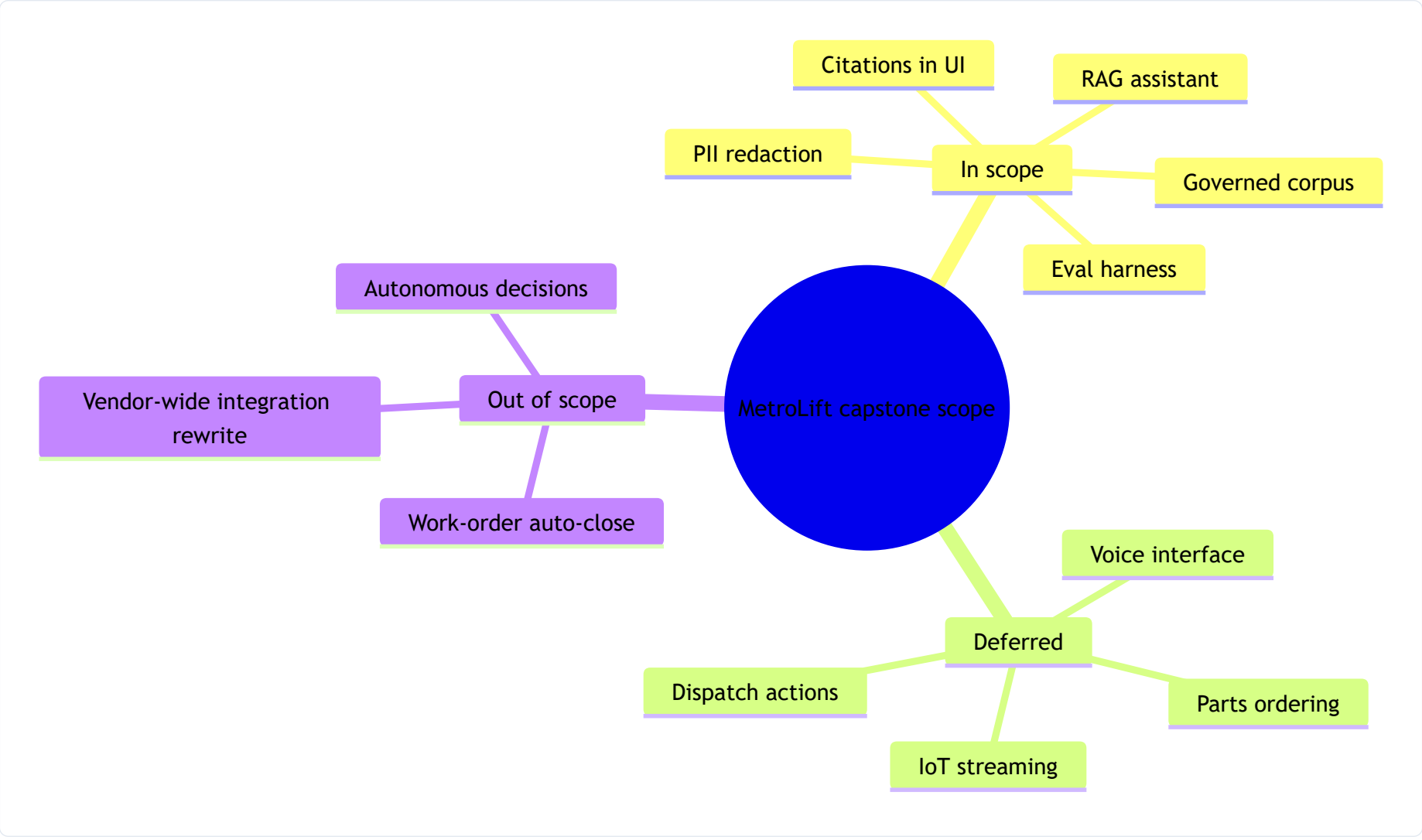
For **Question 2**, Option 1 best fits the capstone because it delivers a clear consumer outcome in **10 weeks** and still includes real governance controls. The right deferred list is concrete: automated dispatch recommendations, parts ordering, multilingual voice, and real-time IoT streams; direct writes back into work-order systems sit one step further, fully out of scope. Those are not 'nice to have' omissions; they are scope boundaries that preserve buildability.

For **Question 3**, a strong trio is **median time to confident next step (< 5 min)**, **citation coverage (≥ 90%)**, and **bulletin freshness (< 24 h)**. Together they force decisions about chunking and retrieval quality, ingestion cadence, access-controlled corpus curation, and UI evidence display. If you picked only latency, you could accidentally build a fast system that is ungrounded and untrustworthy.

For **Question 4**, **PII in incident notes at 8.3%** should change the design immediately because it affects ingestion, storage, access policy, and auditability. Latency can be tuned later within reason; feature ambition can be cut. Sensitive data is different. Once exposed in an index, cleanup is painful and credibility is gone.

For **Question 5**, removing citations would weaken the project even if it improved latency. The primary user need is a **grounded next step with visible evidence**, especially when a bulletin overrides a manual. A 2-second answer without proof is less useful than a 3-second answer with source pages, and it damages the AI-native justification that made RAG worth building at all.

A second visual in the packet summarized the actual scope call Priya proposed:



What transfers beyond MetroLift

If your team says 'AI-native' before it can name a primary consumer, a measurable user task, and the evidence that simpler tools fail, what are you actually committing to build?

- **Start** with the use case brief as a contract test: real consumer, real pain, buildable output.
- **Separate** business, AI, and platform goals so failures stay diagnosable instead of blending into one vanity metric.
- **Use** a small metric set with targets, owners, and evidence sources; seven sharp metrics beat seventeen decorative ones.

- **Cut** scope by naming deferred and out-of-scope items explicitly, especially when an agent-like story is trying to smuggle in unbuildable complexity.
- **Invest** early in design-changing mitigations like PII controls, access rules, and grounded citations; these are architecture decisions, not polish.
- **Prefer** an honest assistant-first platform over a fake-production agent demo that cannot be governed, evaluated, or operated.

Decision Points

Option 1. Approve a narrow assistant-first RAG scope for maintenance manuals and incident bulletins, with governed retrieval, citations, and human escalation.

- **Expected impact:** This delivers a demonstrable AI-native platform in 10 weeks while directly serving field technicians who need grounded answers during elevator outages.
- **Key risk:** The narrower corpus may miss edge-case answers, making coverage gaps visible until more documents are onboarded.
- **Evidence to confirm:** Pilot users achieve at least 75% answer usefulness, 90% citation coverage, and p95 end-to-end latency under 3.5 seconds on the top 25 service questions.
- **Evidence to kill:** If the top technician questions are mostly structured lookups from existing CMMS tables, retrieval adds little value and the AI-native case collapses.

Option 2. Replace RAG with a search-and-dashboard workflow over structured tickets plus keyword search over PDFs.

- **Expected impact:** This cuts implementation complexity and governance burden, and may satisfy supervisors who mainly need counts, trends, and document access.
- **Key risk:** It fails the frontline use case if technicians still spend minutes stitching together procedure steps from scattered manuals during active incidents.

- **Evidence to confirm:** Task completion time drops below 2 minutes for common fault lookups without generative answers, and user interviews show low demand for synthesized responses.
- **Evidence to kill:** If users require cross-document synthesis with grounded evidence from manuals, service bulletins, and parts notes, plain search will not meet the consumer need.

Option 3. Expand the scope to an agent-like operations copilot that recommends parts orders, dispatch actions, and automated work-order updates.

- **Expected impact:** This promises higher business upside and a more impressive demo by linking retrieval with actions across service systems.
- **Key risk:** It introduces decision authority, change-management, and safety risks that the team cannot govern or validate within the capstone timeline.
- **Evidence to confirm:** Only pursue this if action APIs, approval workflows, and risk controls already exist and can be reused with strict human sign-off.
- **Evidence to kill:** If approvals, audit trails, and rollback paths must be invented from scratch, the project becomes an unfinished platform rewrite.

Analysis Walkthrough

The tempting option is the biggest one. MetroLift's VP of Service asked for a 'copilot for field operations,' and that phrase pulled the team toward agent workflows, dispatch actions, and automated updates. But the exhibit numbers point somewhere else: 68% of technician delay came from hunting across PDFs, scanned bulletins, and model-specific procedures, while only 14% came from missing dashboard summaries. The pain is retrieval and grounding, not autonomous action.

Start with the contract test: is there a real consumer, a real problem, and a buildable output? Yes, but only for one slice. Priya Nandakumar, staff data engineer, had one clear primary consumer: field technicians on escalated outages. A secondary consumer existed too: service supervisors reviewing recurring faults. That split matters because the technicians need evidence-backed answers in the moment, while supervisors mostly need trend reporting. One platform can serve both eventually, but one capstone

should not pretend both are equal.

Now the AI-native justification. A non-AI dashboard can summarize outage counts and mean time to repair. It cannot reliably answer, 'For a MetroLift MRL-220 with controller firmware 5.4, after error E217 and a door motor replacement, which bulletin changed the reset sequence and what is the safe verification step?' That query spans unstructured manuals, bulletins, and model notes. The requirement for visible evidence and grounded context is exactly where RAG earns its keep. If the questions were mostly filter-and-group over structured CMMS data, I would kill the AI layer immediately. The case gives the opposite signal.

Option 1 wins because it aligns scope with the measured need. The proposed in-scope corpus is 38,400 maintenance manual pages, 2,100 service bulletins, and 14 months of incident summaries. That is enough heterogeneity to justify governed retrieval without exploding ingestion work. The decision to require citations is not decorative; it is the operating control that lets MetroLift trade a bit of latency for trust. Targets in the style of the DeltaLift example from the Lesson 2 video fit here: p95 latency under 3.5 seconds, answer groundedness above 0.85 on the eval set, and citation coverage above 90%.

Option 2 is credible but weaker. It is the right answer only if evidence shows users just need lookup plus navigation. The table says otherwise: the August shadow study found median troubleshooting time at 11 minutes 40 seconds, and technicians opened 4.7 documents per incident before taking action. A keyword search front end might shave some of that, but it still leaves synthesis on the user. For supervisors, fine. For field technicians standing in a stalled building lobby, not enough.

Option 3 should be rejected now, not because it is a bad future state, but because the risk register changes category when the system starts proposing actions. Safety, access control, approval routing, and auditability go from important to release-blocking. The module's ambition-versus-buildability tradeoff is the whole case: a capstone that claims autonomous recommendations without proving retrieval quality, governance, and rollback is performing realism rather than building it.

The metric design is where weak projects usually collapse. MetroLift originally had 17 candidate metrics on one slide, mixing adoption, retrieval quality, latency, and cost. Priya cut it to seven owned metrics split into business, AI, and platform buckets. That separation is the right move because a good answer rate can rise while freshness quietly fails, or latency can improve while citation coverage drops. One number cannot tell that story.

I would pick **Option 1: narrow assistant-first RAG**. The tradeoff I am accepting is lower feature ambition now in exchange for a platform that can actually be defended: governed corpus, explicit access rules, measurable retrieval quality, and honest deferred scope. The result is less flashy than an agent demo, but much more credible as an AI-native data platform.

The sharpest lesson is what MetroLift should have seen earlier: scope boundaries are not paperwork. They are architecture decisions. Once Priya deferred automated dispatch, parts ordering, multilingual voice input, and real-time IoT streaming, the architecture got simpler, the risk register got more honest, and the success criteria became testable instead of aspirational.